HypG v2.0.1: Normative Specification

A Module-LWE KEM, Symmetric AEAD, and Stateless Hash-Based Signature Scheme

Yellow Paper — Algorithms, Wire Formats, and Conformance Contracts

Workspace 2.0.0; KEM/Symm/Hash 2.0.0; DS 2.0.1

HypG Project

Independent research prototype

October 2026

Abstract

This document is the normative specification of the HypG research prototype. It defines the byte-level encodings, algorithms, domain separators, API contracts, and frozen test-vector inventory for the Module-LWE KEM, the BLAKE3 hash/XOF/keyed-hash layer, the ChaCha20 plus keyed-BLAKE3 authenticated-encryption scheme (one-shot and streaming), and the stateless hash-based signature scheme. The workspace version is 2.0.0; the signature wire/spec revision is separately identified as **2.0.1** because of two changes: the explicit one-byte PRF/F domain tags and FIPS-205-style promotion of odd WOTS+ L-tree nodes.

The document is normative for behaviour, not for security. It does not assert IND-CCA2, UF-CMA, SUF-CMA, or AEAD security for the exact constructions. It does not claim NIST Category-5 security, FIPS conformance, certification, constant-time machine code, or side-channel resistance. In case of disagreement, the Rust implementation and the frozen KAT files take precedence over this prose.

Keywords: specification; Module-LWE; Fujisaki–Okamoto; implicit rejection; BLAKE3; ChaCha20; Encrypt-then-MAC; FORS; WOTS+; XMSS; hypertree; safe Rust; known-answer tests.

Document status. Normative technical report for the v2.0 artifact. It is not peer-reviewed, not an audit, and not a security certification.

Contents

1 Scope, status, and normative precedence **4**
- 1.1 Document identity . . . 4
- 1.2 Normative language . . . 4
- 1.3 Precedence . . . 4
- 1.4 Out of scope . . . 4

2 Notation and encodings **4**
- 2.1 Bytes and integers . . . 4
- 2.2 Rings and vectors . . . 5
- 2.3 The CBD distribution . . . 5

3 Core primitive: CA-PRF **5**
- 3.1 State and round function . . . 5
- 3.2 CA-PRF definition . . . 6

4 Hash, XOF, and KDF layer **6**
4.1 Functions . . . 6
4.2 Framing requirements . . . 7
4.3 Streaming hasher . . . 7

5 Polynomial encodings and compression **7**
5.1 Message encoding . . . 7
5.2 Compression . . . 7
5.3 Packing . . . 8

6 KEM specification **8**
6.1 Parameters and wire sizes . . . 8
6.2 Matrix and vector expansion . . . 8
6.3 Key generation . . . 8
6.4 Encapsulation . . . 9
6.5 Decapsulation . . . 9
6.6 Wire layout . . . 10
6.7 Correctness lemmas . . . 10
6.8 Caller semantics . . . 10
6.9 Malformed inputs . . . 11

7 AEAD specification **11**
7.1 Parameters . . . 11
7.2 Key schedule . . . 11
7.3 One-shot sealing and opening . . . 11
7.4 Streaming state machine . . . 12
7.5 Failure rules . . . 12

8 Signature specification **12**
8.1 Parameters and sizes . . . 12
8.2 Address structure . . . 12
8.3 Tweakable hash functions . . . 13
8.4 Key generation . . . 13
8.5 Randomizer, digest, and split . . . 14
8.6 FORS . . . 14
8.7 WOTS+ . . . 15
8.8 XMSS . . . 15
8.9 Hypertree . . . 16
8.10 Signing and verification . . . 16
8.11 Encoding . . . 17
8.12 DS 2.0.1 revision . . . 17

9 Domain registry **17**

10 API contracts **18**
10.1 KEM . . . 18
10.2 Hash and AEAD . . . 18
10.3 Signatures . . . 18
10.4 Secret-lifetime contract . . . 18

11 Known-answer vectors **18**

12 Validation matrix **19**

13 Conformance and change control **19**

14 Security status and non-claims **20**

A Constant inventory **20**

B Byte-offset quick reference **21**

C CLI and error codes **21**

D Comparison with FIPS 203 and FIPS 205 **21**

Artifact	Code version	Wire/spec revision	Frozen vectors
Workspace	2.0.0	—	—
KEM	2.0.0	KEM v2.0	`kats/v2.0.0.txt`
Symm / AEAD	2.0.0	Symm v2.0	`kats-symm/v2.0.0.txt`
Hash API	2.0.0	Hash v2.0	`kats-hash/v2.0.0.txt`
Signatures	2.0.0	**DS v2.0.1**	`kats-ds/v2.0.1.txt`

Table 1: Version identity. The literal `HYP-G/v07/...` strings are frozen domain separators, not a statement that the current parameter set is v0.7.

1 Scope, status, and normative precedence

1.1 Document identity

1.2 Normative language

The key words MUST, MUST NOT, REQUIRED, SHALL, SHOULD, SHOULD NOT, MAY, and OPTIONAL are to be interpreted as described in RFC 2119 [1] when they appear in this document. In particular:

- A conforming implementation MUST produce the bytes specified for every algorithm in this document.
- A verifier MUST reject a malformed encoding as specified; it MUST NOT silently accept an alternative encoding.
- A caller MUST satisfy the caller-side preconditions (fresh seeds, fresh encapsulation messages, unique nonces, bounded signing volume, and secret storage).

1.3 Precedence

The final arbiters of behaviour are:

1. the Rust code in `crates/`;
2. the active frozen KAT files in `crates/hypg-kat/`;
3. the normative algorithms and tables in this document.

If the prose and the code disagree, the code and KATs win; the prose is then defective and must be corrected. This is intentional: the specification is a research artifact, and silent divergence between paper and implementation is a bug.

1.4 Out of scope

This document does not specify a complete protocol. It does not define key storage, RNG construction, nonce assignment, message framing, replay protection, identity binding, key confirmation, key rotation, or authorization. Those are caller responsibilities. The document also does not provide a security proof; see Section 14.

2 Notation and encodings

2.1 Bytes and integers

Definition 2.1 (Byte string)**.** A byte is an element of $\{0, \ldots, 255\}$. A byte string is a finite sequence of bytes; $\{0,1\}^n$ denotes bit strings of length n, and $\{0,1\}^*$ finite bit strings.

Definition 2.2 (Little-endian encoding)**.** For an integer x with $0 \le x < 2^t$, $\mathrm{LE}_t(x)$ is the $t/8$-byte encoding of x with the least-significant byte first. For example, $\mathrm{LE}_{32}(1) = 01\,00\,00\,00$.

Unless stated otherwise, every multi-byte integer in this specification is little-endian. Byte ranges are half-open: $x[a : b]$ denotes bytes a through $b-1$. The symbol $\|$ denotes concatenation. No implicit padding is ever inserted.

2.2 Rings and vectors

Definition 2.3 (Ring)**.** Let

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{256}+1), \qquad q = 4096 = 2^{12}.$$

An element $a \in \mathcal{R}_q$ is represented by its coefficient vector $(a_0, \ldots, a_{255})$ with $a_i \in [0, q)$. Addition and subtraction are coefficient-wise modulo q.

Definition 2.4 (Negacyclic multiplication)**.** For $a, b \in \mathcal{R}_q$,

$$(a * b)_k = \sum_{\substack{i+j=k \\ 0 \le i,j < 256}} a_i b_j - \sum_{\substack{i+j=k+256 \\ 0 \le i,j < 256}} a_i b_j \pmod q.$$

The implementation evaluates the two sums directly with an integer accumulator and reduces modulo q with a 12-bit mask. The loop bounds are public. For module rank $k = 4$, vectors are written $a = (a_0, \ldots, a_{k-1})$ and $\langle a, b \rangle = \sum_{i=0}^{k-1} a_i b_i$.

2.3 The CBD distribution

Definition 2.5 (CBD(2))**.** On 128 uniformly random bytes, let b_0, b_1, b_2, b_3 be the four successive bits for a coefficient. Define

$$v = (b_0 + b_1) - (b_2 + b_3).$$

The coefficient is v mapped to $[0, q)$ by adding q and masking.

Lemma 2.6 (CBD moments)**.** *For* CBD(2)*, the support is* $\{-2, -1, 0, 1, 2\}$ *and*

$$\Pr[-2] = 1/16, \quad \Pr[-1] = 4/16, \quad \Pr[0] = 6/16, \quad \Pr[1] = 4/16, \quad \Pr[2] = 1/16.$$

The mean is 0 *and the variance is* 1*.*

Proof. The distribution is the difference of two independent Binomial$(2, 1/2)$ variables. Its probability generating function is $(1/4 + 1/2z + 1/4z^2)(1/4 + 1/2z^{-1} + 1/4z^{-2})$, whose coefficients are the probabilities above. The mean is the difference of the two binomial means, and the variance is the sum of the two variances $2 \cdot (1/2)(1/2) = 1/2$. □

3 Core primitive: CA-PRF

3.1 State and round function

Definition 3.1 (CA state and rule)**.** A CA state is a vector $S \in (\mathbb{Z}_{2^{64}})^{32}$, serialized as 32 little-endian 64-bit words (256 bytes). A CA rule is a vector $R \in (\mathbb{Z}_{2^{64}})^{8}$, serialized as eight little-endian 64-bit words (64 bytes). The public constants are

$$\begin{aligned} \mathrm{RC_BASE} &= 0x9E3779B97F4A7C15, \\ \mathrm{RC_STRIDE} &= 0xBF58476D1CE4E5B9, \\ \mathrm{DIFF_MUL} &= 0xD6E8FEB86659FD93. \end{aligned}$$

Definition 3.2 (One CA round)**.** For round index $t \in [0, 32)$ and word index $i \in [0, 32)$, compute, with all arithmetic modulo 2^{64} and ROTL a 64-bit left rotation:

$$
\begin{aligned}
rc &= \mathrm{RC_BASE} + (t+1) \cdot \mathrm{RC_STRIDE},\\
a &= S_i, \quad b = S_{(i+1) \bmod 32}, \quad c = S_{(i+5) \bmod 32}, \quad d = S_{(i+17) \bmod 32},\\
rk &= R_{i \bmod 8} \oplus rc \oplus (i \cdot \mathrm{RC_STRIDE}),\\
t_1 &= a \oplus \mathrm{ROTL}(b, 13) \oplus \mathrm{ROTL}(c, 7),\\
t_2 &= (t_1 \wedge d) \oplus ((\neg t_1) \wedge b),\\
t_3 &= t_2 + (rk \oplus (i + t)),\\
t_4 &= t_3 \oplus \mathrm{ROTL}(t_3, 29),\\
t_5 &= (t_4 \cdot \mathrm{DIFF_MUL}) \oplus \mathrm{ROTL}(d, 11),\\
S_i' &= t_5 \oplus \mathrm{ROTL}(S_{(i+16) \bmod 32}, 17) \oplus rc.
\end{aligned}
$$

The round is applied 32 times.

All loops have public bounds. The temporary array is zeroized after each round. The implementation contains no secret-dependent branch or index in the round.

3.2 CA-PRF definition

Definition 3.3 (CA-PRF)**.** For a 32-byte seed x, public domain d, and output length L:

$$
\begin{aligned}
\mathit{init} &= \mathsf{XOF}(x \,\|\, d \,\|\, \texttt{"-init"}, 256),\\
\mathit{rule} &= \mathsf{XOF}(x \,\|\, d, 64),\\
S_0 &= \mathrm{LE}_{64}^{-1}(\mathit{init}),\\
S_{32} &= \mathrm{Round}^{32}(S_0; \mathit{rule}),\\
\mathit{out} &= \mathsf{XOF}(d \,\|\, \mathrm{LE}_{64}(S_{32}) \,\|\, d \,\|\, \texttt{"-out"}, L).
\end{aligned}
$$

Remark 3.4 (Domain truncation). The implementation copies at most 59 bytes of d before `"-init"` and at most 60 bytes before `"-out"`. Every protocol domain in Section 9 is shorter than these limits, so no truncation occurs. The rule expansion uses the full d.

Remark 3.5 (Security role of the CA). The final BLAKE3 XOF is the security-relevant expander. If BLAKE3 is a secure XOF or random oracle, the output is pseudorandom regardless of the CA. The CA is a deterministic mixing and domain-separation stage, not an independently proven hardness assumption.

Lemma 3.6 (Prefix consistency)**.** *For fixed (x, d) and $L_1 \le L_2$, the first L_1 bytes of* CA-PRF(x, d, L_2) *equal* CA-PRF(x, d, L_1).

Proof. The state and all preceding stages are independent of L. The final XOF is prefix-consistent by definition. □

4 Hash, XOF, and KDF layer

4.1 Functions

Definition 4.1 (Hash API)**.** Let $\mathsf{H}(x)$ be unkeyed BLAKE3-256, $\mathsf{H}_k(x)$ keyed BLAKE3-256 with 32-byte key k, and $\mathsf{XOF}(x, L)$ the first L bytes of the BLAKE3 XOF. Define

$$
\begin{aligned}
\mathrm{expand}(x, d, L) &= \mathsf{XOF}(x \,\|\, d, L),\\
\mathrm{kdf}(d, \mathit{parts}, L) &= \mathsf{XOF}(d \,\|\, \mathit{parts}_0 \,\|\, \mathit{parts}_1 \,\|\, \cdots, L),\\
\mathrm{derive}(c, k) &= \mathrm{BLAKE3.derive_key}(c, k).
\end{aligned}
$$

To avoid a proliferation of symbols, H is overloaded: it denotes the unkeyed one-argument hash in the KEM/hash layer and the keyed four-argument tweakable hash in the DS layer; the argument count and context disambiguate.

BLAKE3 is the hash primitive of the suite [5]. The KEM uses expand only through CA-PRF. The KEM shared-secret derivation uses kdf with the domain `HYP-G/v07/ss` and the fixed-length parts $(m, ct, \mathsf{H}(pk))$. The rejection derivation uses the fixed-length parts $(fake, ct)$.

Lemma 4.2 (XOF prefix property)**.** $\mathsf{H}(x)$ *equals the first 32 bytes of* $\mathsf{XOF}(x, 64)$.

Proof. This is the BLAKE3 root-bytes property and is pinned by the repository tests for empty, short, chunk-edge, and multi-chunk inputs. □

4.2 Framing requirements

Neither expand nor kdf inserts separators or lengths. A caller MUST ensure that its part sequence is injective. This specification uses fixed-length parts for every KDF call. A new caller with variable-length parts MUST add explicit lengths or use `derive_key`.

4.3 Streaming hasher

`Hasher` supports `new`, `new_keyed`, `update`, `finalize32`, and `finalize_xof`. The first 32 bytes of `finalize_xof` equal `finalize32`. The implementation uses a fixed 54-level chaining-value stack and does not allocate.

5 Polynomial encodings and compression

5.1 Message encoding

Definition 5.1 (Encode/Decode)**.** For a 32-byte message m, let

$$\mathsf{Encode}(m)_i = 2048 \cdot ((m_{\lfloor i/8 \rfloor} \gg (i \bmod 8)) \wedge 1).$$

For $v \in \mathcal{R}_q$, let

$$\mathsf{Decode}(v)_i = ((((v_i + 1024) \bmod q) \gg 11) \wedge 1)\,,$$

with the bits packed least-significant first.

5.2 Compression

Definition 5.2 (10-bit and 5-bit compression)**.** For a coefficient $c \in [0, q)$:

$$\begin{aligned}
\mathrm{Compress}_{10}(c) &= ((c + 2) \gg 2) \wedge 0x3FF,\\
\mathrm{Decompress}_{10}(c) &= (c \wedge 0x3FF) \ll 2,\\
\mathrm{Compress}_{5}(c) &= ((c + 64) \gg 7) \wedge 0x1F,\\
\mathrm{Decompress}_{5}(c) &= (c \wedge 0x1F) \ll 7.
\end{aligned}$$

Lemma 5.3 (Residual ranges)**.** *For* $c \in [0, q)$*, write* $\Delta_1 = \mathrm{Decompress}_{10}(\mathrm{Compress}_{10}(c)) - c \pmod q$ *and* $\Delta_2 = \mathrm{Decompress}_{5}(\mathrm{Compress}_{5}(c)) - c \pmod q$*. Interpreted in the signed range,*

$$\Delta_1 \in \{-1, 0, 1, 2\}, \qquad \Delta_2 \in \{-63, -62, \ldots, 64\}.$$

Proof. Write $c = 4a + r$ with $0 \le r < 4$. Then $\mathrm{Compress}_{10}(c) = a$ for $r \le 1$ and $a + 1$ for $r \ge 2$, giving $\Delta_1 = -r$ or $4 - r$. For the 5-bit case, write $c = 128a + r$ with $0 \le r < 128$; the quotient is a for $r < 64$ and $a + 1$ for $r \ge 64$, giving $\Delta_2 = -r$ or $128 - r$. □

Symbol	Value	Meaning
n	256	ring degree
k	4	module rank; dimension 1024
q	$2^{12} = 4096$	modulus
η	2	CBD(2) for s, e, r, e_1, e_2
m	32 bytes	FO message; one bit per coefficient
pk	1568 bytes	$seed_A(32) \,\|\|\, \mathsf{Pack}_{12}(b)(1536)$
ct	1440 bytes	$\mathsf{Pack}_{10}(c_1)(1280) \,\|\|\, \mathsf{Pack}_5(c_2)(160)$
sk	1632 bytes	$seed(32) \,\|\|\, fake(32) \,\|\|\, pk$
ss	32 bytes	XOF output

Table 2: KEM parameters and sizes.

5.3 Packing

Definition 5.4 (12-bit packing)**.** Two coefficients $c_0, c_1 \in [0, 4095]$ are packed into three bytes:

$$b_0 = c_0 \wedge 0xFF, \quad b_1 = ((c_0 \gg 8) \wedge 0x0F) \vee ((c_1 \wedge 0x0F) \ll 4), \quad b_2 = (c_1 \gg 4) \wedge 0xFF.$$

The 10-bit and 5-bit packers use a little-endian bitstream: coefficient i occupies bits $[i \cdot w, (i + 1) \cdot w)$ for $w \in \{10, 5\}$.

Lemma 5.5 (Packing bijectivity)**.** $\mathsf{Pack}_{12}/\mathsf{Unpack}_{12}$*,* $\mathsf{Pack}_{10}/\mathsf{Unpack}_{10}$*, and* $\mathsf{Pack}_5/\mathsf{Unpack}_5$ *are inverse bijections on their fixed-size arrays.*

Proof. The 12-bit packing is a bijection because the three bytes contain exactly the 24 bits of the two 12-bit values in a fixed order. The 10-bit and 5-bit packers write each coefficient into a fixed bit range of a little-endian bitstream of exactly $256w$ bits; no overlapping writes and no unused prefix bits occur. □

6 KEM specification

6.1 Parameters and wire sizes

6.2 Matrix and vector expansion

Definition 6.1 (ParseMatrix)**.** The CA-PRF output of length $4 \cdot 4 \cdot 384 = 6144$ bytes is read in row-major order. Entry (i, j) is the 384-byte block at offset $(4i + j) \cdot 384$, interpreted as 256 12-bit coefficients by Unpack_{12}.

Definition 6.2 (ParseCBDVector)**.** A 512-byte CA-PRF output is split into four 128-byte blocks; each block is mapped through CBD(2) to one polynomial.

6.3 Key generation

Algorithm 1: $\mathsf{KeyGen}(seed)$

1: Input: a 32-byte seed with at least 256 bits of entropy.

2: $seed_A \leftarrow$ CA-PRF($seed$, `"HYP-G/v07/seed-a"`, 32).

3: $A \leftarrow$ ParseMatrix(CA-PRF($seed_A$, `"HYP-G/v07/poly-a"`, 6144)).

4: $s \leftarrow$ ParseCBDVector(CA-PRF($seed$, `"HYP-G/v07/sec-s"`, 512)).

5: $e \leftarrow$ ParseCBDVector(CA-PRF($seed$, `"HYP-G/v07/sec-e"`, 512)).

6: For $i = 0, \ldots, 3$: $b_i \leftarrow \sum_{j=0}^{3} A_{i,j} * s_j + e_i$ in $\mathcal{R}_q$.

7: $pk \leftarrow seed_A \,\|\, \mathsf{Pack}_{12}(b_0) \,\|\, \cdots \,\|\, \mathsf{Pack}_{12}(b_3)$.

8: $fake \leftarrow$ CA-PRF($seed$, `"HYP-G/v07/fake"`, 32).

9: $sk \leftarrow seed \,\|\, fake \,\|\, pk$.

10: Return (pk, sk). Best-effort erase intermediate secret buffers.

6.4 Encapsulation

Algorithm 2: $\mathsf{Encaps}(pk, m)$

1: Input: a 1568-byte public key and a 32-byte FO message m. The caller MUST supply a fresh CSPRNG output for m on every call.

2: Parse pk as $(seed_A, b_0, \ldots, b_3)$.

3: $A \leftarrow$ ParseMatrix(CA-PRF($seed_A$, `"HYP-G/v07/poly-a"`, 6144)).

4: $rseed \leftarrow \mathsf{H}(m \,\|\, pk \,\|$ `"HYP-G/v07/r-seed"`$)$.

5: $r \leftarrow$ ParseCBDVector(CA-PRF($rseed$, `"HYP-G/v07/rand-r"`, 512)).

6: $e_1 \leftarrow$ ParseCBDVector(CA-PRF($rseed$, `"HYP-G/v07/rand-e1"`, 512)).

7: $e_2 \leftarrow$ CBD(CA-PRF($rseed$, `"HYP-G/v07/rand-e2"`, 128)).

8: For $i = 0, \ldots, 3$: $c_{1,i} \leftarrow \sum_{j=0}^{3} A_{j,i} * r_j + e_{1,i}$.

9: $c_2 \leftarrow \sum_{i=0}^{3} b_i * r_i + e_2 + \mathsf{Encode}(m)$.

10: $ct \leftarrow \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,0})) \,\|\, \cdots \,\|\, \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,3})) \,\|\, \mathsf{Pack}_{5}(\mathrm{Compress}_{5}(c_2))$.

11: $ss \leftarrow \mathsf{XOF}($`"HYP-G/v07/ss"` $\,\|\, m \,\|\, ct \,\|\, \mathsf{H}(pk), 32)$.

12: Return (ct, ss).

6.5 Decapsulation

Algorithm 3: $\mathsf{Decaps}(sk, ct)$

1: Parse sk as $(seed, fake, pk)$.

2: Parse pk and expand A as in encapsulation.

3: $s \leftarrow$ ParseCBDVector(CA-PRF($seed$, `"HYP-G/v07/sec-s"`, 512)).

4: Unpack and decompress ct into (c_1, c_2).

5: $v \leftarrow c_2 - \sum_{i=0}^{3} s_i * c_{1,i}$.

6: $m' \leftarrow \mathsf{Decode}(v)$.

7: Recompute ct' exactly as in Algorithm 2 from (pk, m').

8: $ok \leftarrow$ ct_eq(ct', ct) (full scan, returns 1 on equality and 0 otherwise).

9: $pk_hash \leftarrow \mathsf{H}(pk)$.

10: $ss_{\mathrm{ok}} \leftarrow \mathsf{XOF}($`"HYP-G/v07/ss"` $\,\|\, m' \,\|\, ct \,\|\, pk_hash, 32)$.

11: $ss_{\mathrm{rej}} \leftarrow \mathsf{XOF}($`"HYP-G/v07/reject"` $\,\|\, fake \,\|\, ct, 32)$.

12: $ss \leftarrow$ ct_select($ok, ss_{\mathrm{ok}}, ss_{\mathrm{rej}}$).

13: Return ss. The function never returns an error.

Offset	Length	Field in pk
0	32	$seed_A$
32	384	packed b_0
416	384	packed b_1
800	384	packed b_2
1184	384	packed b_3

Offset	Length	Field in ct
0	320	packed compressed $c_{1,0}$
320	320	packed compressed $c_{1,1}$
640	320	packed compressed $c_{1,2}$
960	320	packed compressed $c_{1,3}$
1280	160	packed compressed c_2

Table 3: KEM public-key and ciphertext layouts. The secret key is $seed(32) \,\|\, fake(32) \,\|\, pk(1568)$.

6.6 Wire layout

6.7 Correctness lemmas

Lemma 6.3 (KEM noise identity)**.** *For a ciphertext produced by Algorithm 2 and unpacked by Algorithm 3, before compression effects,*

$$c_2 - \langle s, c_1 \rangle = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle.$$

Proof. Expand $b = As + e$ and $c_1 = A^T r + e_1$, then use associativity and commutativity of multiplication in $\mathcal{R}_q$. The mixed terms $\langle As, r \rangle$ and $\langle s, A^T r \rangle$ are the same sum over the same index pairs. □

Lemma 6.4 (Compression residual identity)**.** *Let $\Delta_{1,i}$ be the residual on $c_{1,i}$ and Δ_2 the residual on c_2. Then*

$$v' = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle + \Delta_2 - \langle s, \Delta_1 \rangle.$$

Proof. Substitute $c'_{1,i} = c_{1,i} + \Delta_{1,i}$ and $c'_2 = c_2 + \Delta_2$ into $v' = c'_2 - \langle s, c'_1 \rangle$ and apply Lemma 6.3. □

Lemma 6.5 (Deterministic re-encryption)**.** *If $\mathsf{Decode}(v') = m$, then Algorithm 3 recomputes $ct' = ct$ and selects ss_{ok}.*

Proof. The expansions, PKE encryption, compression, and packing are deterministic functions of their inputs. With $m' = m$, every intermediate value equals the encapsulation value, including the fixed-length packed ciphertext. The equality test therefore returns 1. □

Failure probability. The working variance model is

$$\mathrm{Var}[\langle e, r \rangle] = 1024, \quad \mathrm{Var}[\langle s, e_1 \rangle] = 1024, \quad \mathrm{Var}[e_2] = 1,$$

$$\mathrm{Var}[\langle s, \Delta_1 \rangle] = 1280, \quad \mathrm{Var}[\Delta_2] = 1365.25,$$

for a total of 4694.25 and $\sigma \approx 68.5$. Rounding up to $\sigma \approx 70$ gives a working margin of approximately 14.6σ against the 1024 decision boundary. This is a heuristic, not a formal tail bound. No analytic 2^{-128} failure bound is claimed, and the 32-trial unit test is a regression tripwire.

6.8 Caller semantics

The 32-byte `coins` argument of the API is m, not a domain-separated auxiliary nonce. It is embedded in c_2 and is the input to the FO randomness generator. It MUST be fresh and uniformly random for every encapsulation under a given public key. The KEM has no internal RNG; key generation, encapsulation, and decapsulation are pure deterministic functions of their inputs.

Parameter	Value
Master key	32 bytes
Encryption subkey	32 bytes
MAC subkey	32 bytes
Nonce	12 bytes
Tag	32 bytes
ChaCha20 block	64 bytes, 20 rounds
Counter	32 bits, initial 0
Maximum message	2^{38} bytes (2^{32} blocks)

Table 4: AEAD parameters.

6.9 Malformed inputs

The public-key and ciphertext parsers are infallible fixed-size wrappers. They do not validate that a public key was produced by Algorithm 1, and they do not reject non-canonical algebraic structure beyond unpacking 12-bit fields. A malformed but correctly sized input cannot cause an out-of-bounds access in safe Rust; decapsulation is total. It can of course produce a useless shared secret.

7 AEAD specification

7.1 Parameters

7.2 Key schedule

$$k_e = \text{derive}(\texttt{"HYP-G/v07/symm-enc"}, K), \qquad k_m = \text{derive}(\texttt{"HYP-G/v07/symm-mac"}, K).$$

ChaCha20 is the IETF 20-round ARX stream cipher of RFC 8439 [6]. The master key is not used directly for ChaCha20 or for the MAC.

7.3 One-shot sealing and opening

Algorithm 4: Seal(K, N, AD, M)

1: Require $|C| = |M|$ and $|M| \le 2^{38}$; otherwise erase C and T and return $\perp$.

2: $(k_e, k_m) \leftarrow$ key schedule.

3: $C \leftarrow M$; apply ChaCha20 with key k_e, nonce N, and counter 0.

4: $T \leftarrow \text{KeyedBLAKE3}(k_m, N \,\|\, AD \,\|\, C \,\|\, \text{LE}_{64}(|AD|) \,\|\, \text{LE}_{64}(|C|))$.

5: Return (C, T).

Algorithm 5: Open(K, N, AD, C, T)

1: Require $|P| = |C|$ and $|C| \le 2^{38}$; otherwise return $\perp$.

2: $(k_e, k_m) \leftarrow$ key schedule.

3: $T' \leftarrow \text{KeyedBLAKE3}(k_m, N \,\|\, AD \,\|\, C \,\|\, \text{LE}_{64}(|AD|) \,\|\, \text{LE}_{64}(|C|))$.

4: If ct_eq$(T', T) \neq 1$, return $\perp$ without writing P.

5: $P \leftarrow C$; apply ChaCha20 with key k_e, nonce N, and counter 0.

6: Return P.

The length suffix is mandatory even when AD or C is empty. The tag covers the ciphertext, not the plaintext.

Condition	Sealer	Opener
length mismatch	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
counter exhausted	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
length overflow	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
finish after poison	erase tag, $\bot$	erase plaintext, $\bot$
finish length mismatch	—	erase plaintext, $\bot$
tag mismatch	—	erase plaintext, $\bot$
success	write tag, $\top$	validate, $\top$

Table 5: Streaming AEAD failure rules. $\top$ denotes `Some(())` and $\bot$ denotes `None`.

7.4 Streaming state machine

`StreamSealer` and `StreamOpener` maintain: a 64-byte keystream block, a partial-block offset, a 64-bit next-counter value, a running keyed-BLAKE3 state, the AD length, the ciphertext length, and a failure flag. `new` absorbs $N \,\|\, AD$. Sealer `update` copies plaintext, XORs the next keystream bytes, adds the chunk length using checked addition, and absorbs the ciphertext. Opener `update` copies ciphertext, XORs tentatively, adds the length, and absorbs the ciphertext.

Proposition 7.1 (Streaming identity)**.** *For every chunking of the same (K, N, AD, M), the streaming sealer emits the same (C, T) as Algorithm 4. For every chunking of the same (K, N, AD, C), the streaming opener accepts exactly when Algorithm 5 accepts and returns the same plaintext.*

Proof sketch. The subkeys, initial MAC state, and length counters are identical. The keystream state preserves the exact 64-byte block and offset across calls; by induction on the chunk sequence, the concatenated XOR bytes equal the one-shot ChaCha20 keystream with counters $0, 1, \ldots$. The running MAC absorbs exactly $N \,\|\, AD \,\|\, C$ and then the two length suffixes. The opener performs the same operations, with the additional invariant that tentative plaintext is zeroized on failure. □

7.5 Failure rules

A poisoned stream never recovers. The opener's plaintext is tentative until `finish` returns $\top$; callers MUST NOT parse, display, execute, or transmit it before that point.

8 Signature specification

8.1 Parameters and sizes

The construction follows the SPHINCS+/SLH-DSA family [8, 4]; XMSS is standardised in RFC 8391 [7]. HypG is not wire-compatible with either standard.

The sizes are:

$$\begin{aligned} \text{FORS unit} &= 32 + 9 \cdot 32 = 320, \\ \text{FORS total} &= 35 \cdot 320 = 11{,}200, \\ \text{XMSS layer} &= 8 + 67 \cdot 32 + 4 \cdot 32 = 2{,}280, \\ \text{hypertree} &= 17 \cdot 2{,}280 = 38{,}760, \\ \mathit{sig} &= 32 + 11{,}200 + 38{,}760 = 49{,}992. \end{aligned}$$

8.2 Address structure

Definition 8.1 (ADRS)**.** An address is 32 bytes:

$$\text{layer}(4) \,\|\, \text{tree}(12) \,\|\, \text{type}(4) \,\|\, f_{20}(4) \,\|\, f_{24}(4) \,\|\, f_{28}(4),$$

Symbol	Value
n	32 bytes (BLAKE3-256)
h	68
d	17 layers
h'	4 (16 leaves per XMSS tree)
k_{FORS}	35
a	9 (512 FORS leaves per tree)
w	16
ℓ	67 (64 message + 3 checksum digits)
pk_{sig}	64 bytes
sk_{sig}	128 bytes
sig	49,992 bytes

Table 6: Signature parameters.

Type	Name	f_{20}	f_{24}	f_{28}
0	`T_FORS`	FORS tree	FORS leaf	zero
1	`T_WOTS_HASH`	keypair	chain	absolute step
2	`T_WOTS_PK`	keypair	L-tree height	pair index
3	`T_TREE`	FORS tree or zero	node height	node index
4	`T_ROOT`	defined, unused	defined, unused	defined, unused

Table 7: Address types and field assignments.

all integers little-endian. The 12-byte tree field holds a 96-bit integer; valid hypertree addresses use at most 68 bits and zero the high 28 bits.

8.3 Tweakable hash functions

All three functions use keyed BLAKE3-256 with the public seed *pubseed* as the 32-byte key:

$$\begin{aligned}
\mathsf{PRF}(pubseed, x, A) &= \mathrm{KeyedBLAKE3}(pubseed, 01 \,\|\, x \,\|\, A),\\
F(pubseed, A, x) &= \mathrm{KeyedBLAKE3}(pubseed, 02 \,\|\, A \,\|\, x),\\
\mathsf{H}(pubseed, A, L, R) &= \mathrm{KeyedBLAKE3}(pubseed, A \,\|\, L \,\|\, R).
\end{aligned}$$

The first two inputs are 65 bytes; the third is 96 bytes. The tags 01 and 02 are the DS 2.0.1 revision. H is untagged but its arity and field order are distinct.

8.4 Key generation

Algorithm 6: DS.KeyGen(esk)

1: $pubseed \leftarrow$ CA-PRF(esk, `"HYP-G/v07/ds-pubseed"`, 32).

2: $prf \leftarrow$ CA-PRF(esk, `"HYP-G/v07/ds-prf"`, 32).

3: $root \leftarrow$ HT.Root($pubseed, esk$), the top XMSS layer at layer 16, tree 0.

4: $pk \leftarrow root \,\|\, pubseed$.

5: $sk \leftarrow esk \,\|\, prf \,\|\, pubseed \,\|\, root$.

6: Return (pk, sk).

8.5 Randomizer, digest, and split

Algorithm 7: PreSign(sk, msg)

1: Parse $(esk, prf, pubseed, root)$ from sk.

2: $stream \leftarrow$ CA-PRF(prf, `"HYP-G/v07/ds-addrnd"`, 64).

3: $R \leftarrow \mathsf{H}(stream \,\|\, \mathsf{H}(msg))$.

4: Compute, for $c \in \{0, 1\}$,

$$md_c = \mathsf{H}(R \,\|\, pubseed \,\|\, root \,\|\, \mathsf{H}(msg) \,\|\, \mathrm{LE}_{32}(c) \,\|\, \texttt{"HYP-G/v07/ds-msghash"}).$$

5: Split the first 387 bits of $md = md_0 \,\|\, md_1$:

$$fors_t = \sum_{b=0}^{8} 2^b \cdot \mathrm{bit}(md, 9t + b), \quad t = 0, \ldots, 34,$$

$$tree = \sum_{b=0}^{67} 2^b \cdot \mathrm{bit}(md, 315 + b), \qquad leaf = \sum_{b=0}^{3} 2^b \cdot \mathrm{bit}(md, 383 + b).$$

6: Return $(R, fors, tree, leaf, esk, pubseed)$.

8.6 FORS

Definition 8.2 (FORS leaf and node)**.** For tree t and leaf l:

$$A = \mathrm{ADRS}(0, 0, 0, t, l, 0), \qquad leaf(t, l) = \mathsf{PRF}(pubseed, esk, A).$$

For a node at height h and pair index p:

$$A = \mathrm{ADRS}(0, 0, 3, t, h, p), \qquad node = \mathsf{H}(pubseed, A, L, R).$$

The FORS public key is

$$\mathrm{FORS.PK} = \mathrm{KeyedBLAKE3}(pubseed, A_{\mathrm{root}} \,\|\, root_0 \,\|\, \cdots \,\|\, root_{34}),$$

where $A_{\mathrm{root}} = \mathrm{ADRS}(0, 0, 0, 0xFFFFFFFF, 0, 0)$ and $root_t$ is the root of tree t. The input is 1152 bytes.

The signature for tree t is the selected leaf plus its nine authentication nodes. Verification recomputes each root from the leaf and path, then compresses the 35 roots. An out-of-range selector returns an all-zero unit; a real root cannot equal that output except with negligible probability.

Lemma 8.3 (FORS reconstruction)**.** *Given a valid FORS signature for indices fors and public seed pubseed,* FORS.Verify *returns the same* FORS.PK *as* FORS.PK *generation.*

Proof. At height h, the verifier uses the current node index $\lfloor fors_t / 2^h \rfloor$, its sibling, and the address $(t, h, \lfloor fors_t / 2^{h+1} \rfloor)$. The signer stored exactly that sibling and the same address. Induction on h therefore reconstructs each tree root. Compression is deterministic. □

8.7 WOTS+

Definition 8.4 (Digits)**.** For a 32-byte message:

$$d_i = (msg_{\lfloor i/2 \rfloor} \gg 4(i \bmod 2)) \wedge 0x0F, \quad i = 0, \ldots, 63.$$

The checksum is

$$c = \sum_{i=0}^{63} (15 - d_i),$$

encoded as three base-16 digits most-significant first: $d_{64} = (c \gg 8) \wedge 0x0F$, $d_{65} = (c \gg 4) \wedge 0x0F$, $d_{66} = c \wedge 0x0F$.

Definition 8.5 (Chain)**.** For a base address with layer, tree, and keypair index, and chain index i:

$$sk_i = \mathsf{PRF}(pubseed, esk, \mathrm{ADRS}(layer, tree, 1, i, 0, 0)).$$

For a segment $[start, start + steps)$:

$$\mathrm{Chain}(x; start, steps, A) = F_{A,\ldots}^{start+steps-1} \circ \cdots \circ F_A^{start}(x),$$

where each F call uses address A with $f_{24} = i$ and $f_{28} = start + s$. The implementation executes all 15 steps and selects branchlessly.

Lemma 8.6 (WOTS chain compatibility)**.** *For $0 \le d \le 15$,*

$$\mathrm{Chain}(\mathrm{Chain}(x; 0, d, A); d, 15 - d, A) = \mathrm{Chain}(x; 0, 15, A).$$

Proof. Both sides apply the same 15 chain steps with absolute addresses $0, \ldots, 14$; composition of functions is associative and the intermediate segment boundary at d does not change the addresses or inputs. □

The L-tree compresses 67 chain ends. At height h and pair index p:

$$A = \mathrm{ADRS}(layer, tree, 2, keypair, h, p), \qquad next_p = \mathsf{H}(pubseed, A, L, R).$$

If the width is odd, the final unpaired node is promoted unchanged. The widths are $67, 34, 17, 9, 5, 3, 2, 1$.

Lemma 8.7 (L-tree determinism)**.** *The L-tree output is a deterministic function of the 67 chain ends and the base address.*

Proof. Each level is a deterministic pairwise map plus a deterministic promotion rule; induction on the width gives the claim. □

8.8 XMSS

An XMSS layer has 16 leaves. For keypair i:

$$\begin{aligned} base &= \mathrm{ADRS}(layer, tree, 0, i, 0, 0), \\ sk &= \mathrm{WOTS.GenSK}(pubseed, esk, base), \\ pk &= \mathrm{WOTS.PKFromSK}(pubseed, sk, base), \\ leaf &= \mathrm{WOTS.LTree}(pubseed, pk, base). \end{aligned}$$

The Merkle root at height h and node index p uses

$$A = \mathrm{ADRS}(layer, tree, 3, 0, h, p), \qquad node = \mathsf{H}(pubseed, A, L, R).$$

The XMSS signature is a WOTS+ signature plus four authentication nodes. Verification recomputes the leaf and climbs with the same parity rule: if the current index is even, the computed node is the left child; otherwise it is the right child.

Lemma 8.8 (XMSS reconstruction)**.** *For a valid XMSS signature, verification returns the same root as XMSS public-key generation.*

Proof. The WOTS+ signature reconstructs the selected leaf by Lemma 8.6, and the authentication path supplies the sibling at every height. Induction on height, with the address and parity rules above, reconstructs the Merkle root. □

8.9 Hypertree

For layer $i = 0, \ldots, 16$:

$$tree_i = tree_idx \gg 4(i+1), \qquad leaf_0 = leaf, \qquad leaf_i = (tree_idx \gg 4i) \wedge 15 \quad (i \geq 1).$$

Layer 0 signs the FORS public key; each higher layer signs the root returned by the layer below. 'HT.Root' is the XMSS root of layer 16, tree 0.

Lemma 8.9 (Hypertree consistency)**.** *For an honestly generated key and signature, hypertree verification returns 'HT.Root'.*

Proof. Induction on the layer index. The base message is the FORS public key. At layer i, XMSS reconstruction gives the next message; after layer 16 the message is the top root, which equals the public-key root by construction. □

8.10 Signing and verification

Algorithm 8: DS.Sign(sk, msg)

1: $(R, fors, tree, leaf, esk, pubseed) \leftarrow$ PreSign(sk, msg).

2: Compute the 35 FORS units by signing each $(fors_t)$.

3: Compute the FORS public key from the units (or from the secret).

4: $m_0 \leftarrow$ FORS.PK.

5: For $i = 0, \ldots, 16$:

   a) $s_i \leftarrow$ XMSS.Sign$(pubseed, esk, i, tree_i, leaf_i, m_i)$.

   b) $m_{i+1} \leftarrow$ XMSS.Verify$(pubseed, i, tree_i, leaf_i, m_i, s_i)$.

6: Serialize R, the 35 FORS units, and the 17 layer signatures, inserting eight zero bytes before each layer's WOTS+ signature.

7: Return the 49,992-byte signature.

Algorithm 9: DS.Verify(pk, msg, sig)

1: If $|sig| \neq 49{,}992$, return `false`.

2: Parse R, $root$, $pubseed$.

3: Recompute the digest and split into FORS indices, tree index, and leaf.

4: Parse the 35 FORS units and compute the FORS public key.

5: For each layer, require the eight reserved bytes to be zero.

6: Climb all 17 XMSS layers in order.

7: Return ct_eq$(root', root) = 1$.

8.11 Encoding

$$sig = R(32) \,\|\, \mathrm{FORS}_{35} \,\|\, \mathrm{layer}_0 \,\|\, \cdots \,\|\, \mathrm{layer}_{16}.$$

Each FORS unit is $32 + 9 \cdot 32 = 320$ bytes. Each layer is

$$\mathrm{zero}(8) \,\|\, \mathrm{WOTS}(67 \cdot 32) \,\|\, \mathrm{auth}(4 \cdot 32) = 2280$$

bytes. The total is $32 + 35 \cdot 320 + 17 \cdot 2280 = 49{,}992$.

8.12 DS 2.0.1 revision

Relative to DS 2.0.0, revision 2.0.1 changes exactly two wire-visible items:

1. PRF prefixes its input with 01; F prefixes its input with 02.
2. The WOTS+ L-tree promotes an unpaired final node unchanged instead of hashing it with itself.

The first change removes the 64-byte structural collision between PRF and F inputs. The second follows the FIPS 205 odd-node rule and changes every WOTS+ L-tree root, every XMSS root, and every signature. The frozen file is `kats-ds/v2.0.1.txt`. The v2.0.0 DS vector file remains historical.

9 Domain registry

Table 8: Domain registry. Every active string is listed; the `-init`/`-out` suffixes are CA-PRF mechanism, not separate rows.

Domain	Construction	Purpose
`HYP-G/v07/seed-a`	CA-PRF(seed, d, 32)	KEM $seed_A$
`HYP-G/v07/sec-s`	CA-PRF(seed, d, 512)	KEM secret s
`HYP-G/v07/sec-e`	CA-PRF(seed, d, 512)	KEM error e
`HYP-G/v07/poly-a`	CA-PRF(seed_A, d, 6144)	matrix A
`HYP-G/v07/rand-r`	CA-PRF(rseed, d, 512)	FO r
`HYP-G/v07/rand-e1`	CA-PRF(rseed, d, 512)	FO e_1
`HYP-G/v07/rand-e2`	CA-PRF(rseed, d, 128)	FO e_2
`HYP-G/v07/fake`	CA-PRF(seed, d, 32)	rejection key
`HYP-G/v07/r-seed`	BLAKE3(m ‖ pk ‖ d)	FO seed
`HYP-G/v07/ss`	XOF(d ‖ m ‖ ct ‖ H(pk),32)	accepted secret
`HYP-G/v07/reject`	XOF(d ‖ fake ‖ ct,32)	rejected secret
`HYP-G/v07/symm-enc`	BLAKE3 derive_key	AEAD enc subkey
`HYP-G/v07/symm-mac`	BLAKE3 derive_key	AEAD MAC subkey
`HYP-G/v07/ds-addrnd`	CA-PRF(prf, d, 64)	randomizer stream
`HYP-G/v07/ds-msghash`	digest suffix	digest separation
`HYP-G/v07/ds-pubseed`	CA-PRF(esk, d, 32)	public hash key
`HYP-G/v07/ds-prf`	CA-PRF(esk, d, 32)	randomizer seed
`HYP-G/v07/kat`	hash KAT only	never protocol use

The CA-PRF mechanism appends `"-init"` and `"-out"` to a domain; those suffixes are not separate registry rows. The retired draft strings `ds-fors`, `ds-wots`, and `ds-tree` are not used.

File	Bytes	SHA-256
kats/v2.0.0.txt	28,514	3b2dde3d...71d4b
kats-symm/v2.0.0.txt	3,662	91b7942f...4208
kats-hash/v2.0.0.txt	923	5fa77803...d9f1
kats-ds/v2.0.1.txt	201,015	e88c1bdf...c813
Total	234,114	

Table 9: Frozen files. The complete hashes are in docs/parameters.md.

10 API contracts

10.1 KEM

```
keygen(seed: &[u8; 32]) -> (PublicKey, SecretKey)
encaps(pk: &PublicKey, coins: &[u8; 32])
    -> (Ciphertext, SharedSecret)
decaps(sk: &SecretKey, ct: &Ciphertext) -> SharedSecret
```

decaps never fails; an invalid ciphertext returns the pseudorandom rejection secret. SecretKey and SharedSecret are not Copy; their Debug is redacted; equality is constant-time; Drop best-effort zeroizes.

10.2 Hash and AEAD

```
hash32, hash64, keyed32, derive32, expand, kdf, Hasher
seal(key, nonce, ad, pt, ct, tag) -> Option<()>
open(key, nonce, ad, ct, tag, pt) -> Option<()>
StreamSealer::new/update/finish
StreamOpener::new/update/finish
```

One-shot open leaves the plaintext buffer untouched on authentication failure. Streaming opening is tentative until finish.

10.3 Signatures

```
ds_keygen(esk: &[u8; 32]) -> (SigPk, SigSk)
ds_sign(sk: &SigSk, msg: &[u8], sig: &mut [u8; 49992])
ds_verify(pk: &SigPk, msg: &[u8], sig: &[u8]) -> bool
mt_sign(sk: &SigSk, msg: &[u8], sig: &mut [u8; SIG_LEN])
```

ds_sign is deterministic in (sk,msg). ds_verify returns false on any length or encoding error. mt_sign is byte-identical to ds_sign and never panics on a lost worker; the lost chunk is recomputed serially.

10.4 Secret-lifetime contract

SecretKey, SharedSecret, SigSk, PreSign, Key, State, Rule, and Hasher redact Debug and best-effort zeroize on drop. This is not volatile erasure. Accessors such as seed_copy, fake_copy, and public_copy return caller-owned copies; the caller must erase them.

11 Known-answer vectors

The complete SHA-256 values are:

Gate	Result	Evidence class
Debug tests	146/146	E1
Release tests	146/146	E1
MSRV 1.70 tests	146/146	E1
Red-team S1–S14	14/14	E2
Failure sweep (release)	20,000/20,000	E2
Full-bit flip sweep (one ct)	11,520/11,520	E2
Lattice lab L1–L5	5/5	E2
Cross-architecture KAT	234,114 bytes identical	E3
Miri KEM	9/9	E3
Miri AEAD	6/6	E3
Miri DS bounds	3/3	E3

Table 10: Validation record. E4 (independent audit) is absent.

```
kats/v2.0.0.txt
3b2dde3d1138c11b54a6880f50ca7b4390071ae53aa94601075f39b2af971d4b
kats-symm/v2.0.0.txt
91b7942ffb4a3c5da8b8b589268af7b7d1594796b428efdc7e26c567c5f04208
kats-hash/v2.0.0.txt
5fa778035ce2e8aa9c89345458ba4da67df4bdbe71a456cbb58026cf9341d9f1
kats-ds/v2.0.1.txt
e88c1bdf54e9390736cd03c1a165616d744a42c48d662880b344c7de3546c813
```

The KEM file contains three deterministic tag groups (00, 42, FF) and a BLAKE3 `"abc"` anchor. The Symm file contains five AEAD cases (empty, one-byte, 127/128-byte, and 1000-byte messages with varied AD). The Hash file contains empty, `"abc"`, and 1025-byte pattern cases plus keyed and derive-key anchors. The DS file contains two cases: an empty message and a 55-byte message, each with a 49,992-byte signature.

The x86-64 and `qemu-aarch64` musl-static KAT outputs are byte-identical across all 234,114 bytes. This checks endianness and integer width on an emulated ARM64 target; it is not native ARM64 validation.

12 Validation matrix

The debug, release, and MSRV suites each contain 108 unit tests, 25 CLI negative integration tests, and 13 doc-tests. Full DS signing is not covered by Miri. The failure sweep and bit-flip sweep are sanity checks, not a formal failure bound. The lattice lab is not a BKZ cost estimate.

```
cargo test --workspace --locked
cargo test --workspace --locked --release
cargo +1.70 test --workspace --locked
cargo run -q -p hypg-redteam
cargo run -q -p hypg-lattice
cargo test -p hypg-kat --lib --locked
```

13 Conformance and change control

An implementation conforms to this document for a given revision only if it reproduces the active KAT files byte-for-byte and satisfies the specified failure and encoding rules. A change is wire-visible if it alters any of:

- a ring parameter, modulus, distribution, packing, compression, or message encoding;
- a domain string, tag byte, hash input order, or ADRS field;
- a DS tree shape, digest split, checksum rule, or reserved byte;
- an AEAD key-derivation context, MAC input, or length limit;
- a frozen KAT file or its interpretation.

Any such change requires a new version identifier, regenerated KATs, an updated registry, a cross-architecture equality run, and a new specification revision. A test, benchmark, or zeroization change that does not alter bytes is not wire-visible, but must still preserve the frozen output.

14 Security status and non-claims

This specification does not prove any security property. The intended analysis depends on Module-LWE hardness, BLAKE3, ChaCha20, the exact FO-style transform, random-oracle behaviour, a negligible decapsulation failure rate, nonce uniqueness, and a bounded signing volume. Published FO analyses [9, 10] do not automatically cover the chosen accepted-secret and rejection derivations. None of these is established by the implementation or by this document.

The KEM has no exact published proof for its shared-secret and rejection derivations; the lattice-estimator has not been run; the signature composition has no exact UF-CMA/SUF-CMA proof or signing-volume bound; the AEAD has no reduction for the exact composition; and no external audit, formal verification, constant-time machine-code proof, or power/EM/fault campaign exists. HypG is a research prototype and MUST NOT be deployed to protect real assets.

A Constant inventory

Table 11: Constant inventory.

Constant	Value
STATE_WORDS	32
STATE_BYTES	256
RULE_WORDS	8
ROUNDS	32
SEED_LEN	32
FAKE_LEN	32
PK_LEN	1568
CT_LEN	1440
SK_LEN	1632
SS_LEN	32
POLY_N	256
MODULE_K	4
MOD_Q	4096
C1_Q	1024
C2_Q	32
CBD_ETA	2
POLY_PACKED_LEN	384
C1_PACKED_LEN	320

Table 11: Constant inventory.

Constant	Value
`C2_PACKED_LEN`	160
`SIG_PK_LEN`	64
`SIG_SK_LEN`	128
`SIG_LEN`	49,992

B Byte-offset quick reference

Object	Offset	Field
sk	0	master seed (32)
sk	32	fake key (32)
sk	64	public key (1568)
sig	0	R (32)
sig	32	FORS units (11,200)
sig	11,232	layer 0 (2,280)
sig	47,712	layer 16 (2,280)

Selected byte offsets. Each layer begins with eight zero bytes, followed by 67 WOTS+ values and four authentication nodes.

C CLI and error codes

The operator CLI is not a production key-management tool. Exit code 0 means success; code 2 means usage or input error; code 3 means authentication failure (bad AEAD tag or bad signature). KEM decapsulation of an invalid ciphertext exits 0 because the library returns a pseudorandom secret rather than an error.

D Comparison with FIPS 203 and FIPS 205

HypG resembles ML-KEM-1024 [2] in module rank and ring degree but uses $q = 4096$, 10-bit c_1 compression, a different FO-style secret derivation, and BLAKE3. It resembles SLH-DSA-256f (FIPS 205) [4] in its tree dimensions but uses BLAKE3, a custom 32-byte address, explicit PRF/F tags, deterministic randomizer derivation, and eight reserved bytes per layer. It is not ML-KEM, not SLH-DSA, and not FIPS-conformant. Matching dimensions is not a security certification. ML-DSA (FIPS 204) [3] is not used by HypG.

References

[1] S. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, March 1997.

[2] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, August 2024.

[3] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, August 2024.

[4] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. FIPS 205, August 2024.

[5] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. Technical report, 2020.

[6] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, June 2018.

[7] A. Hülsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, May 2018.

[8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The SPHINCS+ signature framework. In *ACM CCS 2019*, pages 2129–2146, 2019.

[9] E. Fujisaki and T. Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *CRYPTO 1999*, LNCS 1666, pages 537–554, 1999.

[10] D. Hofheinz, K. Hövelmanns, and E. Kiltz. A modular analysis of the Fujisaki–Okamoto transformation. In *TCC 2017*, LNCS 10677, pages 341–371, 2017.